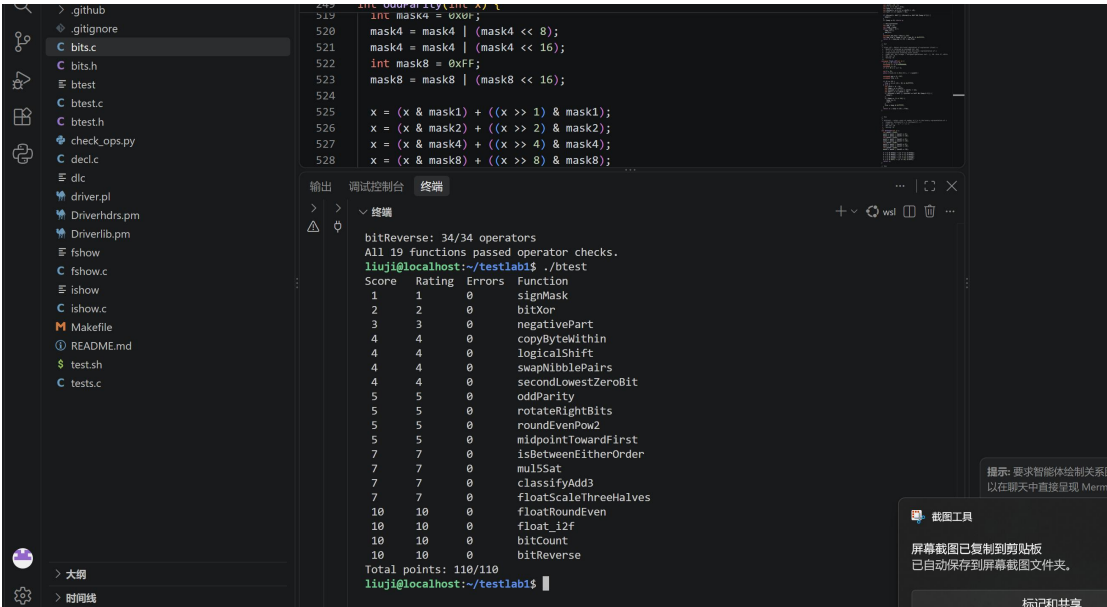
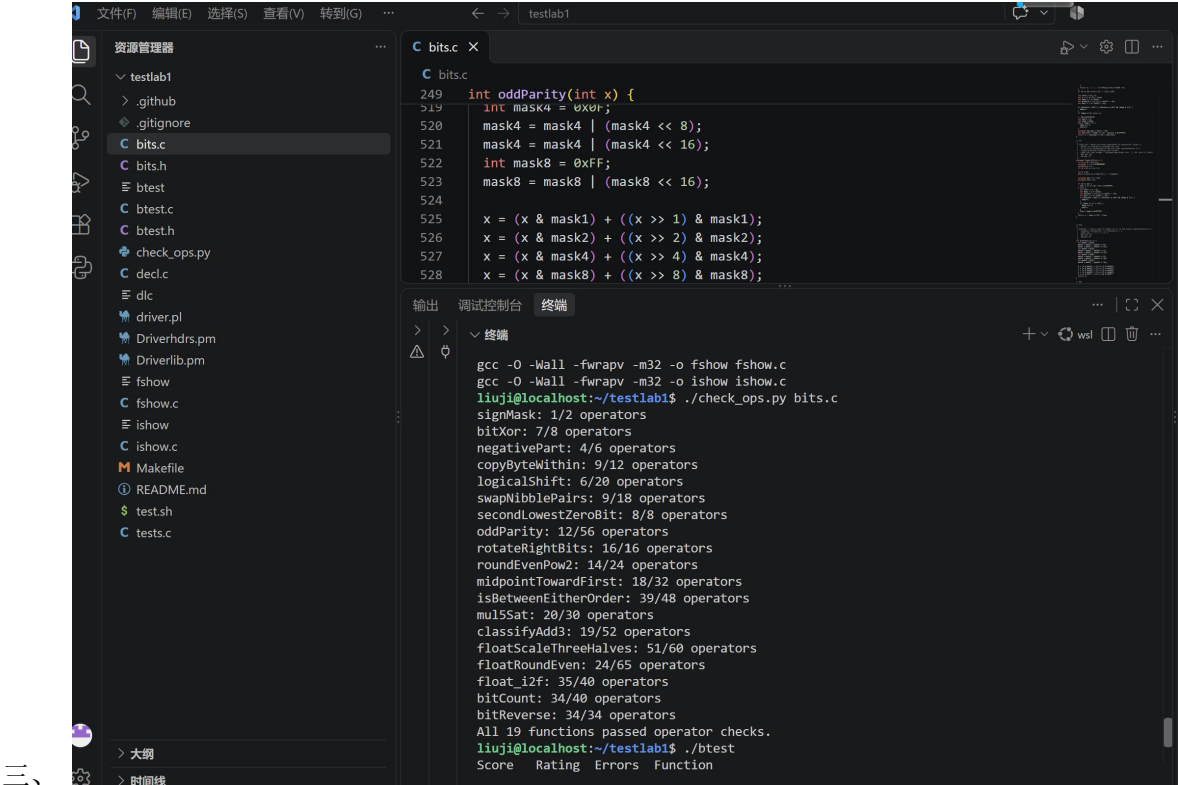


一、Lab1: DataLab 实验报告

二、测试结果截图



三、函数实现思路解析

整数题部分 (P1 - P14, P18, P19)

P1: signMask

- 思路：返回仅最高有效位（符号位）设置的掩码。只需将 1 左移 31 位即可。
- 代码：return 1 << 31;

P2: bitXor

- 思路：仅使用 ~ 和 & 实现异或。利用德摩根定律： $x \oplus y = (x \mid y) \& \sim(x \& y)$ 。由于 | 不可用，将 $x \mid y$ 替换为 $\sim(\sim x \& \sim y)$ 。
- 代码：return $\sim(\sim x \& \sim y) \& \sim(x \& y)$;

P3: negativePart

- 思路：若 $x < 0$ 返回 $-x$ ，否则返回 0。通过 $x \gg 31$ 获取符号位掩码（负数为全 1，正数为全 0），再和 $\sim x + 1$ （即 $-x$ ）做按位与运算。
- 代码：return $(x \gg 31) \& (\sim x + 1)$;

P4: copyByteWithin

- 思路：提取 src 字节，清除 dst 字节区域，然后将提取的字节左移到 dst 位置进行合并。shift = $n \ll 3$ 相当于 $n * 8$ 位。
- 代码：见 bits.c，核心为掩码操作 $(x \& \sim \text{mask}) \mid (\text{extracted} \ll \text{dst_shift})$ 。

P5: logicalShift

- 思路：C 语言的 $\gg$ 是算术右移（高位补符号位）。通过构造掩码 $\sim(((1 \ll 31) \gg n) \ll 1)$ ，将其与 $(x \gg n)$ 进行按位与，清除高位的符号填充位。
- 代码：return $(x \gg n) \& \sim(((1 \ll 31) \gg n) \ll 1)$;

P6:

- 思路：交换每个字节内的高低 4 位。通过移位构造出 0x0F0F0F0F 掩码。提取低 4 位左移 4 位，提取高 4 位右移 4 位，最后按位或合并。
- 代码：利用 mask 拼接和移位实现。

P7:

- 思路：要找 x 中第二低的 0 位。先对 x 取反得到 y，y 中最低的 1 即为 x 中最低的 0。利用 $y \& (\sim y + 1)$ 提取最低位 1，将其消去后再提取最低位 1，即得到结果。
- 代码：思路：y ^= lowest; return y & ($\sim y + 1$);

P8

- 思路：奇偶校验位。要求 1 的个数为偶数时返回 1，奇数时返回 0。采用异或倍增法将

所有位折叠到最后一位，最后取反（符合奇偶定义）。

- 代码：`x ^= x >> 16; ... return !(x & 1);`

P9: rotateRightBits

- 思路：循环右移。注意题目要求 n 可以很大，实际位数需要按 32 取模 ($n \& 31$)。防止移位超过 31 导致未定义行为。左移部分为了对称也应取模。
- 代码：`int n_mod = n & 31; ... return left | right;`

P10: roundEvenPow2

- 思路：四舍五入到 2 的幂。关键在于构造 `add` 值：包含半程基准值 `half`，以及根据第 n 位是否为 1 决定是否额外加 1（偶数舍入规则）。最后加上 `add` 并清除低 n 位。

P11: midpointTowardFirst

- 思路：不溢出的求平均，且中间值向第一个参数 x 靠近。利用 $x \wedge y$ 和 $x \& y$ 组合求平均。通过判断 x 和 y 的符号位及大小关系来决定是否补加 1。

P12: isBetweenEitherOrder

- 思路：判断 x 是否在 a 、 b 之间。由于不能用比较运算符，用 $x \wedge a$ 和 $x \wedge b$ 提取符号差异，利用移位 31 位构造掩码，判断是否满足 $(a \leq x \leq b)$ 或 $(b \leq x \leq a)$ 。

P13: mul5Sat

- 思路：计算 $x * 5$ ，溢出时饱和截断。 $x * 5 = x + (x \ll 2)$ 。通过判断 x 和 $x5$ 的符号位是否改变来检测溢出，若溢出根据符号位返回 `INT_MAX` 或 `INT_MIN`。

P14: classifyAdd3

- 思路：判断 $x+y+z$ 是否溢出。不能使用更宽类型，分两步计算 $x+y$ 和 $+z$ ，分别检测溢出方向和最终溢出符号。

P18: bitCount

- 思路：分治法计算二进制中 1 的个数。构造 `0x55...`, `0x33...` 等掩码，两两合并、四四合并、八八合并，不断将相邻位块相加。

P19: bitReverse

- 思路：反转 32 位整数。经典分治策略，先交换相邻 1 位，再交换相邻 2 位、4 位、8 位、16 位。通过移位掩码实现。注意高位常量需移位拼接生成

浮点题部分 (P15 - P17)

P15: floatScaleThreeHalves

- 思路：计算浮点数 $f * 1.5$ 。提取符号 s 、阶码 exp 和尾数 $frac$ 。处理 NaN 和无穷大直接返回。非规格化数特殊处理。加上隐含 1 后，乘法等价于 $m = m + (m >> 1)$ 。关注舍入：根据最低位判断是否进位，并处理阶码溢出。

P16: floatRoundEven

- 思路：浮点数四舍五入到整数。提取位级表示，判断阶码 E 。处理 $E < 0$ 的情况（绝对值小于 1）。当需要舍弃位时，依据“向偶数舍入”规则（余数大于一半进位，等于一半且保留位末位为 1 时进位）进行舍入，最后重新组装阶码和尾数。保留 -0 的符号。

P17: float_i2f

- 思路：整数转浮点数。先求绝对值，找到最高非零位 E 。构造阶码 $exp = E + 127$ 。若 $E > 23$ ，需要舍入，同样采用“向偶数舍入”规则，若进位导致第 24 位产生，阶码需加 1。最后拼接符号、阶码和尾数。

四、实验总结

- 遇到的难点：

1. 环境的搭建：需要配置 WSL 并在其中安装 32 位编译环境（gcc-multilib 等），第一次接触 WSL 容易在 Windows 目录下编译导致报错。

2. 操作数限制与违禁操作符：如 P8 之前错误地返回了 1 导致奇偶校验逻辑反转；P9 由于移位量没有取模 $n \& 31$ 导致大数移位越界出现未定义行为；P6 使用了大常量超过 0xFF 的常值限制，需要改为位拼接。

3. 浮点数的舍入：浮点数向偶数舍入的规则容易遗漏，尤其是 P16 的 -0 处理和 P17 的进位处理。

- 感受：通过这次实验，我彻底摒弃了对十进制和常规算术运算的依赖，学会了在纯位级视角下进行逻辑推演和算法设计。虽然过程充满挑战，经常因为一个操作符或边界情况 debug 很久，但看到 Total points: 110/110 时非常有成就感。